1. Introduction to Scikit-learn :-

What is Scikit-learn?

Scikit-learn is one of the most popular open-source Python libraries used for Machine Learning (ML). It provides simple and efficient tools for building machine learning models without requiring complex mathematical implementations.

Built on top of NumPy, SciPy, and Matplotlib, Scikit-learn offers a wide range of algorithms for tasks such as classification, regression, clustering, and dimensionality reduction.

Whether you're predicting house prices, detecting spam emails, recognizing handwritten digits, or grouping similar data, Scikit-learn provides easy-to-use tools to accomplish these tasks.

2. History of Scikit-learn :-

Scikit-learn has become one of the most popular machine learning libraries in Python due to its simplicity, efficiency, and strong community support. It was developed to make machine learning accessible to everyone—from beginners to experienced data scientists.

Timeline of Scikit-learn :-

2007 - The Beginning

- Scikit-learn was started as a Google Summer of Code (GSoC) project. It was initiated by David Cournapeau.
- The goal was to create a simple and efficient machine learning library for Python.

2010 - First Public Release

- The first official version (v0.1) was released. It provided several machine learning algorithms for classification, regression, and clustering.
- The library quickly gained popularity because of its simple and consistent API.

2012-2018 - Rapid Growth

- Many new algorithms and utilities were added. Performance improvements made the library faster and more reliable.
- Better documentation and tutorials attracted millions of users.

2019-Present

- Scikit-learn is now one of the world's most widely used machine learning libraries. It is actively maintained by a large open-source community.
- It is widely used in: Education Research Industry Data Science Artificial Intelligence projects

Why Scikit-learn was Developed :-

Before Scikit-learn, implementing machine learning algorithms in Python was difficult and time-consuming. Developers often had to write complex mathematical code from scratch, making machine learning inaccessible to many beginners.

Objectives of Scikit-learn:-

1. Simplify Machine Learning:

Scikit-learn provides ready-to-use machine learning algorithms, allowing users to build models with just a few lines of code.

2. Provide a Consistent API:

All algorithms in Scikit-learn follow a similar interface (`fit()`, `predict()`, `score()`), making it easy to learn and switch between different models.

3. Save Development Time:

Instead of implementing algorithms from scratch, developers can use Scikit-learn's built-in implementations, reducing development time and effort.

4. Support Data Preprocessing:

Real-world data often needs cleaning and preparation before training a model. Scikit-learn includes tools for:

- Handling missing values
- Feature scaling
- Encoding categorical data
- Splitting datasets

Features of Scikit-learn :-

Scikit-learn provides many useful features that make machine learning easy, efficient, and beginner-friendly.

1. Easy to Use:

Simple and consistent API for building machine learning models.

2. Multiple ML Algorithms:

Supports classification, regression, clustering, and more.

3. Data Preprocessing:

Provides tools for scaling, encoding, and handling missing data.

4. Model Evaluation:

Includes metrics like Accuracy, Precision, Recall, and F1-Score.

5. Hyperparameter Tuning:

Optimize models using `GridSearchCV` and `RandomizedSearchCV`.

6. Cross Validation:

Tests model performance on different data splits for better reliability.

Applications of Scikit-learn :-

Scikit-learn is widely used in various industries to solve real-world machine learning problems.

1. Spam Email Detection:

Classifies emails as spam or not spam.

2. House Price Prediction:

Predicts house prices based on various features.

3. Medical Diagnosis:

Helps detect diseases using patient data.

4. Customer Segmentation:

Groups customers based on their behavior and preferences.

5. Recommendation Systems:

Suggests products, movies, or music to users.

6. Fraud Detection:

Identifies suspicious banking or online transactions.

Installing and Importing Scikit-learn

Before using Scikit-learn, you need to install and import the library.

1. Install Scikit-learn:

Use the following command to install Scikit-learn.

```
pip install scikit-learn
```

---

2. Import Scikit-learn:

Import the Scikit-learn library using:

```
import sklearn
```

---

3. Check the Installed Version:

You can verify the installed version using:

```
import sklearn

print(sklearn.__version__)
```

```
pip install scikit-learn
```

```
import sklearn
```

```
import sklearn

print(sklearn.__version__)
```

```
1.6.1
```

Machine Learning Basics :-

Machine Learning (ML) is a branch of Artificial Intelligence (AI) that enables computers to learn from data and make predictions or decisions without being explicitly programmed.

Instead of following fixed rules, machine learning models identify patterns in data and improve their performance through experience.

Types of Machine Learning:

1. Supervised Learning:

In Supervised Learning, the model is trained using **labeled data**, where both the input and the correct output are known.

Examples:

- House Price Prediction
- Spam Email Detection
- Student Result Prediction

Example of Supervised Learning :-

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
```

```
# Load dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Predict
prediction = model.predict(X_test)

print("First 5 Predictions:", prediction[:5])
```

```
First 5 Predictions: [1 0 2 1 1]
```

2. Unsupervised Learning:

In Unsupervised Learning, the model is trained using **unlabeled data**. The goal is to discover hidden patterns or groups within the data.

Examples:

- Customer Segmentation
- Market Basket Analysis
- Image Clustering

Example of Unsupervised Learning :-

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load dataset
X = load_iris().data

# Create model
kmeans = KMeans(n_clusters=3, random_state=42)

# Train model
kmeans.fit(X)

print("Cluster Labels:")
print(kmeans.labels_[:10])
```

```
Cluster Labels:
[1 1 1 1 1 1 1 1 1 1]
```

3. Reinforcement Learning:

In Reinforcement Learning, an agent learns by interacting with its environment. It receives rewards for correct actions and penalties for incorrect actions.

Examples:

- Self-driving Cars
- Robotics
- Game Playing (Chess, Go)

Built-in Datasets :-

Scikit-learn provides several built-in datasets that help beginners learn and practice machine learning without downloading data from external sources.

These datasets are commonly used for classification, regression, and clustering tasks.

Popular Built-in Datasets:

1. Iris Dataset:

The Iris dataset contains measurements of iris flowers and is used for classification tasks.

Target Classes:

- Setosa
- Versicolor
- Virginica

```
from sklearn.datasets import load_iris

iris = load_iris()

print(iris.DESCR)
```

2. Wine Dataset:

The Wine dataset contains the chemical properties of different wine samples. It is mainly used for classification problems.

```
from sklearn.datasets import load_wine

wine = load_wine()

print(wine.DESCR)
```

3. Breast Cancer Dataset :-

This dataset contains medical information used to classify tumors as malignant or benign.

```
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()

print(cancer.DESCR)
```

4. Digits Dataset :-

The Digits dataset contains images of handwritten digits (0–9) and is used for image classification tasks.

```
from sklearn.datasets import load_digits

digits = load_digits()

print(digits.DESCR)
```

Data Preprocessing :-

Data preprocessing is the process of preparing raw data before training a machine learning model. It improves data quality and helps models achieve better accuracy.

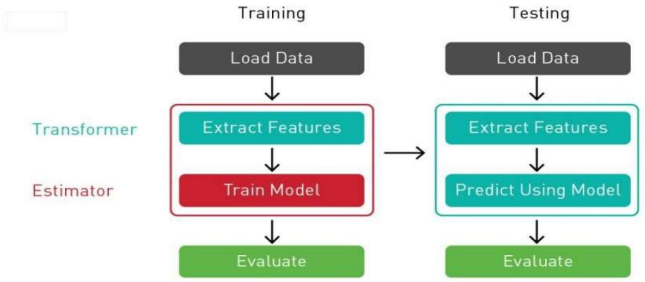
The most common preprocessing techniques in Scikit-learn are:

- Train-Test Split
- Feature Scaling
- Label Encoding
- One-Hot Encoding

1. Train-Test Split:

Train-Test Split divides the dataset into two parts:

- **Training Data:** Used to train the model.
- **Testing Data:** Used to evaluate the model's performance.




```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

X, y = load_iris(return_X_y=True)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training Data Shape:", X_train.shape)
print("Testing Data Shape:", X_test.shape)
```

```
Training Data Shape: (120, 4)
Testing Data Shape: (30, 4)
```

2. Feature Scaling:

Feature Scaling brings all features to a similar range, helping many machine learning algorithms perform better.

```
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import load_iris

X, _ = load_iris(return_X_y=True)

scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)

print(X_scaled[:5])
```

```
[[-0.90068117  1.01900435 -1.34022653 -1.3154443 ]
 [-1.14301691 -0.13197948 -1.34022653 -1.3154443 ]
 [-1.38535265  0.32841405 -1.39706395 -1.3154443 ]
 [-1.50652052  0.09821729 -1.2833891  -1.3154443 ]
 [-1.02184904  1.24920112 -1.34022653 -1.3154443 ]]
```

3. Label Encoding:

Label Encoding converts categorical labels into numerical values.

```
from sklearn.preprocessing import LabelEncoder

colors = ["Red", "Blue", "Green", "Red", "Blue"]

encoder = LabelEncoder()

encoded = encoder.fit_transform(colors)

print(encoded)
```

```
[2 0 1 2 0]
```

4. One-Hot Encoding:

One-Hot Encoding converts categorical values into binary columns.

```
from sklearn.preprocessing import OneHotEncoder
```

```
data = [["Red"], ["Blue"], ["Green"]]

encoder = OneHotEncoder(sparse_output=False)

encoded = encoder.fit_transform(data)

print(encoded)
```

```
[[0. 0. 1.]
 [1. 0. 0.]
 [0. 1. 0.]]
```

Regression Algorithms :-

Regression is a type of supervised machine learning used to predict **continuous numerical values**. It learns the relationship between input features and the target variable to make future predictions.

Examples:

- House Price Prediction
- Stock Price Prediction
- Salary Prediction
- Temperature Forecasting

1. Linear Regression :-

Linear Regression is one of the simplest and most widely used regression algorithms. It finds the best-fit line that represents the relationship between the input features and the target variable.

It is mainly used when the target value is continuous.

Example:

- Predicting the price of a house based on its area.

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
import numpy as np

# Sample data
X = np.array([[1000], [1200], [1500], [1800], [2000]])
y = np.array([200000, 240000, 300000, 360000, 400000])

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create and train model
model = LinearRegression()
model.fit(X_train, y_train)

# Predict
prediction = model.predict([[1600]])

print("Predicted Price:", prediction[0])
```

```
Predicted Price: 320000.0
```

2. Decision Tree Regressor:-

Decision Tree Regressor predicts continuous values by splitting the data into smaller groups based on feature values.

It is useful for capturing non-linear relationships between variables.

Example:

- Predicting crop yield based on rainfall and temperature.

```
from sklearn.tree import DecisionTreeRegressor
import numpy as np
```

```
X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = DecisionTreeRegressor(random_state=42)
model.fit(X, y)

print(model.predict([[6]]))
```

```
[10.]
```

3. Random Forest Regressor :-

Random Forest Regressor combines multiple decision trees to make more accurate and reliable predictions.

It generally performs better than a single decision tree by reducing overfitting.

Example:

- Predicting house prices using multiple features.

```
from sklearn.ensemble import RandomForestRegressor
import numpy as np

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = RandomForestRegressor(random_state=42)
model.fit(X, y)

print(model.predict([[6]]))
```

```
[9.1]
```

Classification Algorithms :-

Classification is a type of supervised machine learning used to predict **categorical values**. It assigns data into predefined classes based on the input features.

Examples:

- Spam Email Detection
- Disease Prediction
- Handwritten Digit Recognition
- Customer Churn Prediction

1. Logistic Regression :-

Logistic Regression is a classification algorithm used to predict binary or multi-class outcomes.

Example:

- Predicting whether an email is Spam or Not Spam.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = LogisticRegression(max_iter=200)
model.fit(X_train, y_train)

# Predict
```

```
prediction = model.predict(X_test)

print(prediction[:5])
```

```
[1 0 2 1 1]
```

2. K-Nearest Neighbors (KNN) :-

KNN classifies data by finding the **K nearest neighbors** and assigning the most common class among them.

Example:

- Classifying flowers based on their measurements.

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier

X, y = load_iris(return_X_y=True)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

print(model.predict([X[0]]))
```

```
[0]
```

3. Decision Tree Classifier :-

Decision Tree Classifier predicts the class by creating a tree-like structure of decisions based on the input features.

Example:

- Loan Approval Prediction.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

X, y = load_iris(return_X_y=True)

model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

print(model.predict([X[0]]))
```

```
[0]
```

4. Support Vector Machine (SVM) :-

SVM separates data into different classes by finding the best decision boundary (hyperplane).

Example:

- Face Recognition.

```
from sklearn.datasets import load_iris
from sklearn.svm import SVC

X, y = load_iris(return_X_y=True)

model = SVC()
model.fit(X, y)

print(model.predict([X[0]]))
```

```
[0]
```

Clustering Algorithms :-

Clustering is a type of **Unsupervised Learning** that groups similar data points into clusters without using labeled data.

Examples:

- Customer Segmentation
- Document Grouping
- Image Segmentation
- Market Analysis

1. K-Means Clustering :-

K-Means is one of the most popular clustering algorithms. It divides the dataset into **K clusters** by grouping similar data points together.

Example:

- Grouping customers based on their purchasing behavior.

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans

# Load dataset
X = load_iris().data

# Create K-Means model
kmeans = KMeans(n_clusters=3, random_state=42)

# Train model
kmeans.fit(X)

# Display cluster labels
print(kmeans.labels_[:10])
```

```
[1 1 1 1 1 1 1 1 1 1]
```

2. DBSCAN:-

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups closely packed data points together and identifies outliers as noise.

Example:

- Detecting unusual or abnormal data points.

```
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering

# Load dataset
X = load_iris().data

# Create model
model = AgglomerativeClustering(n_clusters=3)

# Train model
labels = model.fit_predict(X)

print(labels[:10])
```

```
[1 1 1 1 1 1 1 1 1 1]
```

Model Evaluation :-

Model Evaluation is the process of measuring how well a machine learning model performs on unseen data. It helps us determine whether the model is making accurate predictions and whether it needs improvement.

Different evaluation metrics are used for **Regression** and **Classification** models.

Examples:

- Measuring prediction accuracy
- Comparing multiple models
- Detecting overfitting or underfitting
- Selecting the best-performing model

1. Accuracy Score :-

Accuracy measures the percentage of correctly predicted values out of the total predictions.

Formula:

Accuracy = (Correct Predictions / Total Predictions) × 100

Example:

- If a model predicts 90 out of 100 values correctly, the accuracy is **90%**.

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Train model
model = DecisionTreeClassifier()
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
Accuracy: 1.0
```

2. Confusion Matrix:-

A Confusion Matrix is a table that shows how many predictions were correct and incorrect for each class.

It helps identify:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

Example:

- Evaluating a Spam Detection model.

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, y_pred)

print(cm)
```

```
[[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

3. Mean Squared Error (MSE):-

Mean Squared Error measures the average squared difference between the actual and predicted values.

A **lower MSE** indicates better model performance.

Example:

- Evaluating a house price prediction model.

```
from sklearn.metrics import mean_squared_error
from sklearn.linear_model import LinearRegression
```

```
import numpy as np

X = np.array([[1], [2], [3], [4], [5]])
y = np.array([2, 4, 6, 8, 10])

model = LinearRegression()
model.fit(X, y)

predictions = model.predict(X)

print("MSE:", mean_squared_error(y, predictions))
```

```
MSE: 0.0
```

Cross Validation :-

Cross Validation is a technique used to evaluate the performance of a machine learning model by dividing the dataset into multiple parts (called folds).

Instead of using only one train-test split, the model is trained and tested multiple times using different data splits. This provides a more reliable estimate of the model's performance.

Benefits:

- Reduces overfitting
- Gives a more reliable accuracy score
- Makes better use of the available data

1. K-Fold Cross Validation:-

K-Fold Cross Validation divides the dataset into **K equal parts (folds)**.

- One fold is used for testing.
- The remaining folds are used for training.
- This process repeats until every fold has been used as the test set once.
- The final performance is calculated by taking the average of all scores.

Example:

- If **K = 5**, the dataset is divided into 5 folds, and the model is trained and tested 5 times.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import cross_val_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Create model
model = DecisionTreeClassifier(random_state=42)

# Perform 5-Fold Cross Validation
scores = cross_val_score(model, X, y, cv=5)

print("Scores:", scores)
print("Average Accuracy:", scores.mean())
```

```
Scores: [0.96666667 0.96666667 0.9        0.93333333 1.        ]
Average Accuracy: 0.9533333333333334
```

2. Stratified K-Fold Cross Validation:-

Stratified K-Fold is similar to K-Fold, but it ensures that each fold has approximately the same proportion of each class as the original dataset.

This method is mainly used for **classification problems**.

Example:

- If a dataset contains 70% Class A and 30% Class B, each fold will maintain a similar ratio.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

# Load dataset
X, y = load_iris(return_X_y=True)

# Create model
model = DecisionTreeClassifier(random_state=42)

# Stratified K-Fold
skf = StratifiedKFold(n_splits=5)

scores = cross_val_score(model, X, y, cv=skf)

print("Scores:", scores)
print("Average Accuracy:", scores.mean())
```

```
Scores: [0.96666667 0.96666667 0.9        0.93333333 1.        ]
Average Accuracy: 0.9533333333333334
```

Hyperparameter Tuning :-

Hyperparameter Tuning is the process of finding the best set of parameters for a machine learning model to improve its performance.

Hyperparameters are values that are set **before** training the model. Choosing the right hyperparameters can increase the model's accuracy and reduce overfitting.

Examples of Hyperparameters:

- Number of Neighbors (`n_neighbors`) in KNN
- Maximum Depth (`max_depth`) in Decision Trees
- Number of Trees (`n_estimators`) in Random Forest

1. Grid Search:-

Grid Search tries **all possible combinations** of hyperparameter values and selects the combination that gives the best performance.

Although it is slower, it provides the most accurate result.

Example:

- Finding the best values for `max_depth` and `criterion` in a Decision Tree.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV

# Load dataset
X, y = load_iris(return_X_y=True)

# Create model
model = DecisionTreeClassifier(random_state=42)

# Hyperparameters
params = {
    "max_depth": [2, 3, 4, 5],
    "criterion": ["gini", "entropy"]
}

# Grid Search
grid = GridSearchCV(model, params, cv=5)
grid.fit(X, y)

print("Best Parameters:", grid.best_params_)
print("Best Score:", grid.best_score_)
```

```
Best Parameters: {'criterion': 'gini', 'max_depth': 3}
Best Score: 0.9733333333333334
```

2. Randomized Search:-

Randomized Search selects **random combinations** of hyperparameters instead of trying every possible combination.

It is faster than Grid Search, especially when there are many hyperparameters.

Example:

- Quickly finding good hyperparameter values for a Random Forest model.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import RandomizedSearchCV

# Load dataset
X, y = load_iris(return_X_y=True)

# Create model
model = DecisionTreeClassifier(random_state=42)

# Hyperparameters
params = {
    "max_depth": [2, 3, 4, 5, 6, 7],
    "criterion": ["gini", "entropy"]
}

# Randomized Search
random_search = RandomizedSearchCV(
    model,
    param_distributions=params,
    n_iter=4,
    cv=5,
    random_state=42
)

random_search.fit(X, y)

print("Best Parameters:", random_search.best_params_)
print("Best Score:", random_search.best_score_)
```

```
Best Parameters: {'max_depth': 6, 'criterion': 'entropy'}
Best Score: 0.9533333333333334
```

Model Saving & Loading :-

After training a machine learning model, we often want to save it so that we don't have to train it again every time we use it.

Scikit-learn allows us to save trained models to a file and load them later whenever needed.

Benefits:

- Saves training time
- Reuse trained models
- Easy deployment in real-world applications

1. Saving a Model using Joblib:-

`joblib` is the recommended library for saving Scikit-learn models. It stores the trained model in a file that can be loaded later.

Example:

- Saving a trained Decision Tree model.

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
import joblib

# Load dataset
X, y = load_iris(return_X_y=True)

# Train model
model = DecisionTreeClassifier(random_state=42)
model.fit(X, y)

# Save model
joblib.dump(model, "decision_tree_model.pkl")
```

```
print("Model saved successfully!")
```

```
Model saved successfully!
```

2. Loading a Saved Model:-

A saved model can be loaded using `joblib.load()` and used to make predictions without retraining.

Example:

- Loading the previously saved Decision Tree model.

```
import joblib

# Load saved model
loaded_model = joblib.load("decision_tree_model.pkl")

# Make prediction
prediction = loaded_model.predict([[5.1, 3.5, 1.4, 0.2]])

print("Predicted Class:", prediction)
```

```
Predicted Class: [0]
```

Conclusion :-

Scikit-learn is one of the most popular and beginner-friendly machine learning libraries in Python. It provides simple and efficient tools for building, training, and evaluating machine learning models.

Key Takeaways:

- Easy to learn and use
- Supports Regression, Classification, and Clustering
- Includes powerful preprocessing and evaluation tools
- Offers built-in datasets for practice
- Helps build accurate and efficient machine learning models